Machine Learning: Prediction and Classification Models

Contents

1 Regression Modeling **2**

2 Classification Models **12**

3 Neural Network Classification (PyTorch) **17**

4 Neural Network Regression (PyTorch) **21**

5 Code and Implementation **24**

1 Regression Modeling

In this section, we analyze a supervised regression problem using the Auto MPG dataset. The objective is to predict a car's fuel efficiency (measured in miles per gallon, mpg) based on vehicle characteristics.

Exploratory Data Analysis (2.1)

Missing Values

Feature	Missing Values
mpg	0
cylinders	0
displacement	0
horsepower	6
weight	0
acceleration	0
model_year	0
origin	0

Table 1: Number of missing values per feature

We first examined missing values in the dataset. The only feature containing missing values is `horsepower`, with 6 missing entries. These values are handled in the code under Section 2.2 (Dataset and Preprocessing) using mean imputation within the preprocessing pipeline.

Distribution of the Target Variable (MPG)

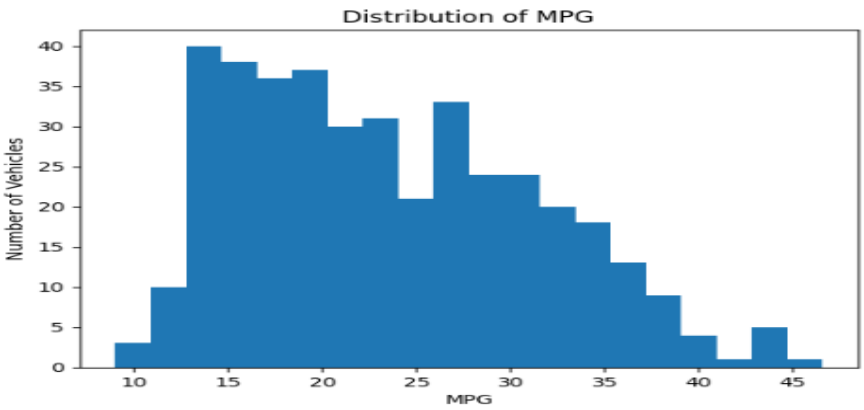

Figure 1: Distribution of MPG

The histogram illustrates the distribution of the target variable MPG.
The x-axis represents MPG values, while the y-axis represents the number of vehicles in each MPG range. The histogram shows that most vehicles have MPG values roughly between 12 and 30. Only a small number of cars achieve very high fuel efficiency above 35 MPG.

Relationships Between Features and the Target

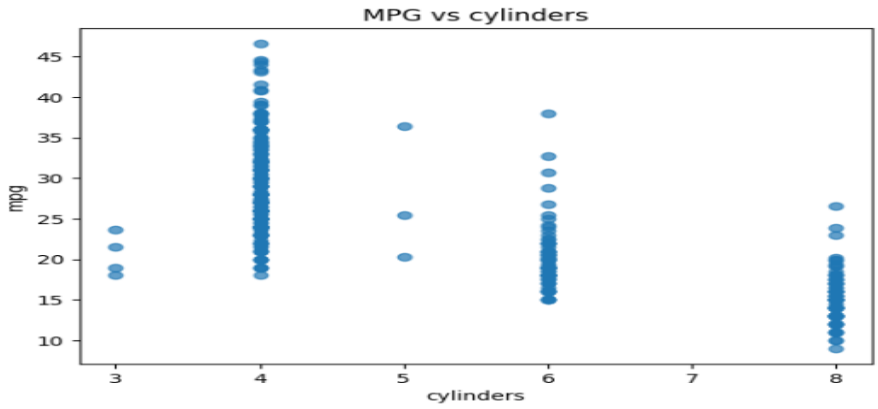

Figure 2: MPG vs Cylinders

Vehicles with more cylinders tend to have lower MPG values.
Cars with 8 cylinders tend to exhibit the lowest fuel efficiency, while vehicles with 4 cylinders tend to achieve higher MPG. Vehicles with 6 cylinders tend to fall in between these two groups.

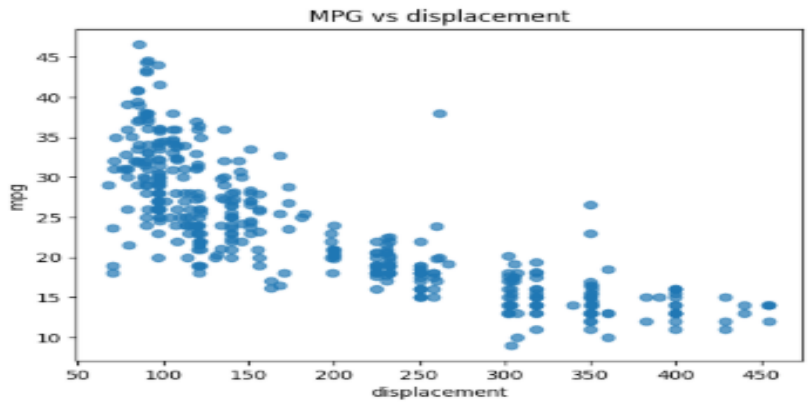

Figure 3: MPG vs Displacement

MPG decreases as engine displacement increases. This indicates that vehicles with larger engines tend to consume more fuel.

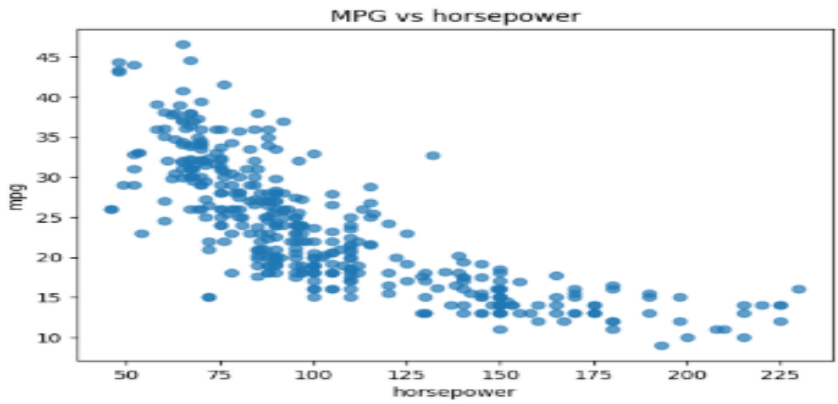

Figure 4: MPG vs Horsepower

MPG decreases as horsepower increases, indicating that more powerful engines tend to consume more fuel.

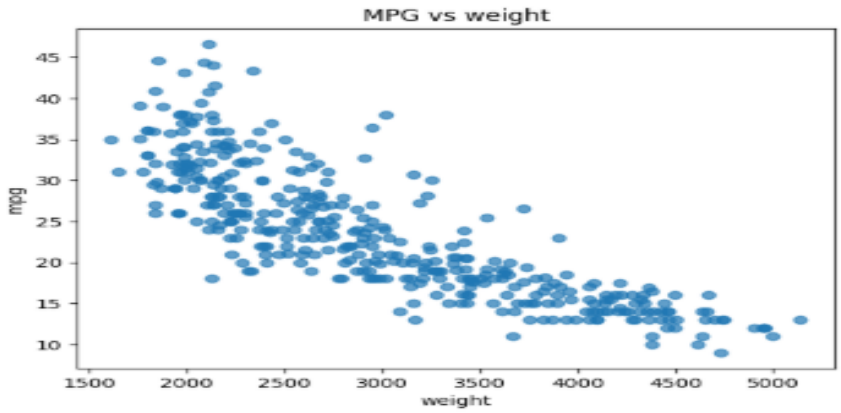

Figure 5: MPG vs Weight

MPG decreases as vehicle weight increases, indicating that heavier vehicles tend to consume more fuel.

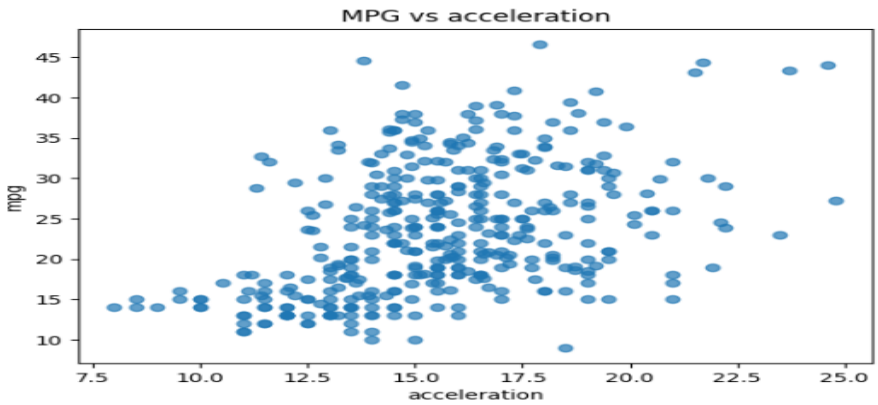

Figure 6: MPG vs Acceleration

Acceleration shows a weaker relationship with MPG compared to other variables.
Although MPG may slightly increase as acceleration increases, the points are quite scattered, indicating that the relationship between the variables is not very strong.

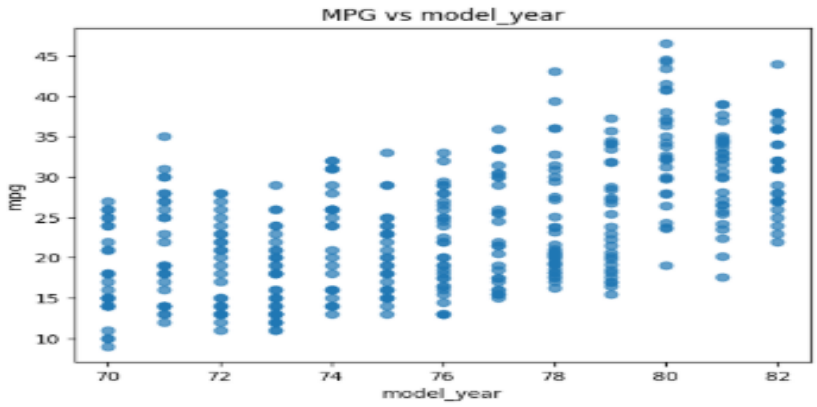

Figure 7: MPG vs Model Year

MPG tends to increase as the model year increases, indicating that newer vehicles generally have better fuel efficiency.

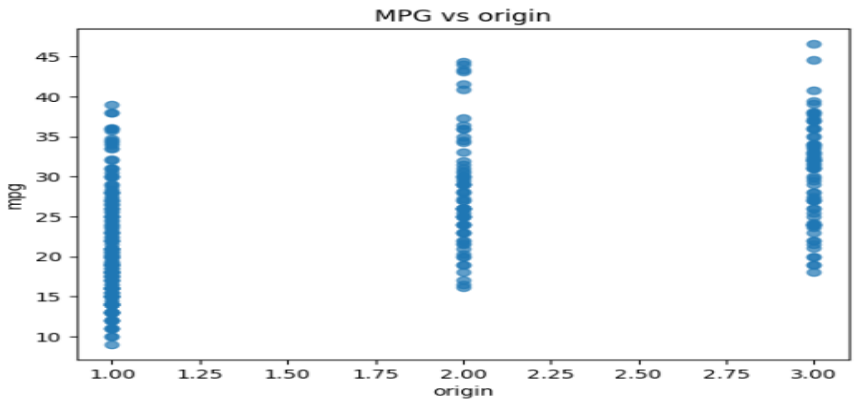

Figure 8: MPG vs Origin

MPG varies across different origins. Vehicles from origin 1 tend to have lower MPG values, while vehicles from origins 2 and 3 generally achieve higher MPG values.

The variable car_name represents the name of the vehicle and is mainly used as an identifier. Since it does not provide meaningful numerical information, it was not included in the relationship analysis.

Correlations Among Features

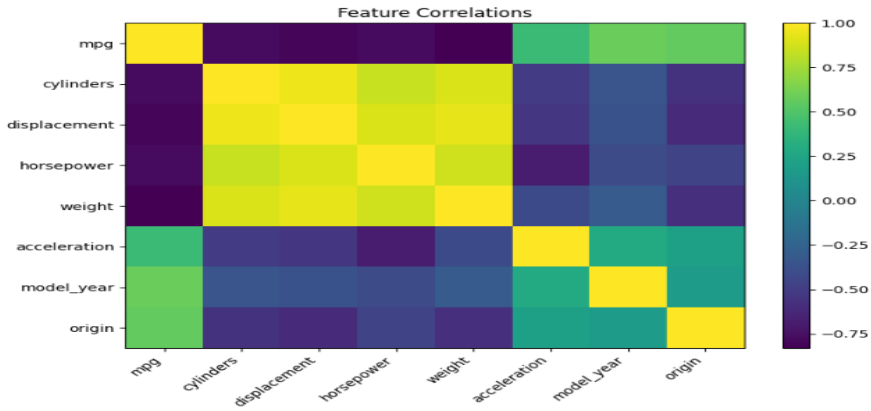

Figure 9: Feature Correlations

Higher values in the heatmap indicate stronger positive correlations, while lower values indicate negative correlations.
Strong correlations exist among weight, cylinders, displacement, and horsepower. MPG shows negative correlations with these variables and a positive correlation with model year and origin.

Dataset and Preprocessing (2.2)

Features Used for Prediction: The column *car_name* was excluded since it is a textual feature and is not useful for regression models. Therefore, the remaining features (*cylinders, displacement, horsepower, weight, acceleration, model_year, origin*) were used for prediction, while *mpg* was used as the target variable.

Handling Missing Values: The feature *horsepower* contains 6 missing values in the dataset. These values were replaced with the average value of the feature.

Feature Standardization: The numerical features (*cylinders, displacement, horsepower, weight, acceleration, model_year*) were standardized so that they are on a similar scale. This is important for models that rely on distances between data points, such as KNN, and also helps gradient-based optimization methods such as SGD converge more efficiently.

The feature *origin* represents a categorical variable, therefore it was not standardized and was encoded using one-hot encoding.

Linear Regression Baseline (2.3)

We begin our analysis with a linear regression model as a baseline, representing the simplest reasonable approach to the problem.

The feature-target relationship plots (e.g., MPG vs weight) were already presented in Section 2.1 as part of the exploratory data analysis.
Based on these plots, some features such as weight, horsepower, and displacement show an approximately linear relationship with MPG, while other variables exhibit weaker or less clear relationships. Therefore, the linearity assumption of linear regression appears to be reasonable as a baseline model for this problem.

The linear regression model was trained on the training set and evaluated on the validation set.
The evaluation metrics on the validation set are:
MSE = 12.898, RMSE = 3.591, MAE = 2.796, $R^2 = 0.814$

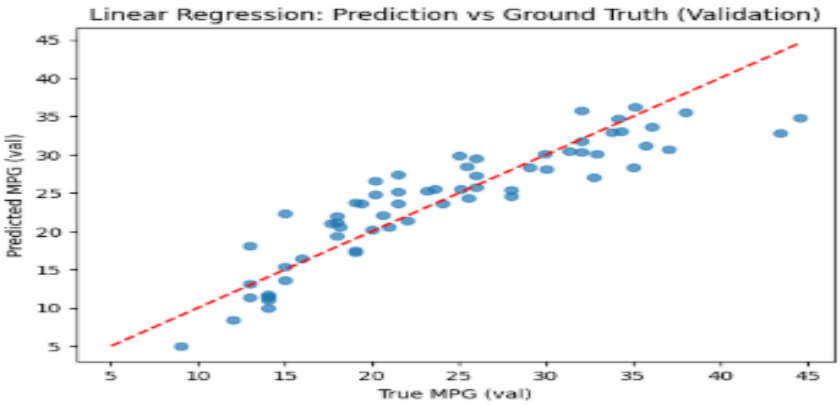

Figure 10: Linear Regression: Prediction vs Ground Truth (Validation)

The prediction vs. ground truth plot shows that a large portion of the predictions lie close to the diagonal line, indicating that the model captures the general trend in the data. However, larger deviations from the line can be observed for very low and very high MPG values, meaning that the model is less accurate at the extremes. In contrast, for mid-range MPG values the predictions appear closer to the diagonal line.

Polynomial Regression and Model Complexity (2.4)

To increase the flexibility of the linear regression model, polynomial regression models with degrees ranging from 1 to 10 were trained.
For each polynomial degree, the model was evaluated using the Mean Squared Error (MSE) on both the training set and the validation set.
MSE was chosen as the evaluation metric since it is a standard loss function for regression problems and penalizes larger prediction errors more strongly.

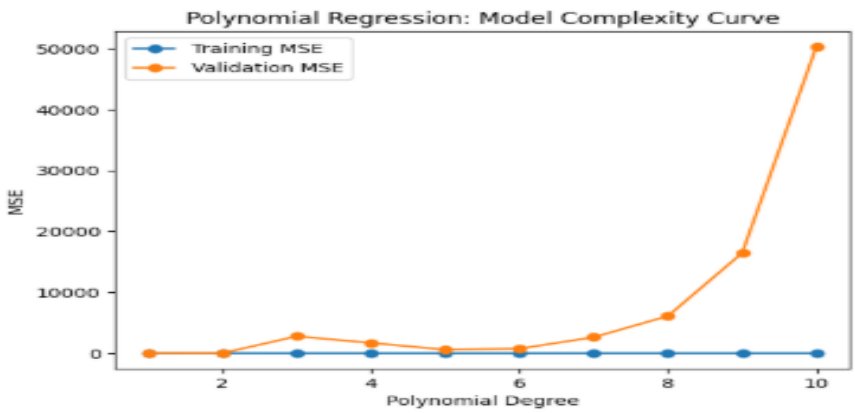

Figure 11: Model complexity curve showing training and validation MSE as a function of polynomial degree.

The figure shows that for low polynomial degrees the model achieves relatively low training and validation errors.
In our results, there is no clear evidence of underfitting, since both the training and validation errors are relatively low for the lower polynomial degrees. Typically, underfitting would be reflected by high errors on both the training and validation sets when the model is too simple.

While the training error remains very low as the polynomial degree increases, the validation error increases sharply for higher polynomial degrees. This indicates overfitting, where the model fits the training data too closely and therefore its ability to generalize to new data is reduced.

As the polynomial degree increases (i.e., as model complexity increases), the bias of the model typically decreases because the model can fit the training data more closely. However, higher model complexity also increases the variance, making the model more sensitive to the training data. High variance can lead the model to fit noise in the training data, which results in overfitting. As observed in our results, this occurs for higher polynomial degrees.

The polynomial degree was selected based on the lowest validation MSE since the validation set provides an estimate of the model's performance on unseen data.
Although the figure shows very similar validation errors for degrees 1 and 2, the exact numerical values computed in the code indicate that degree 2 achieves the lowest validation MSE. Therefore, degree 2 was selected as the final polynomial regression model.

K-Nearest Neighbors (KNN) Regression (2.5)

To analyze a non-parametric regression model using K-Nearest Neighbors (KNN), the performance of the model was evaluated for multiple values of k.
For each value of k, the model was trained on the training set and evaluated using the Mean Squared Error (MSE) on both the training set and the validation set. The best value of k was selected based on the lowest validation MSE.
MSE was chosen as the evaluation metric since it is a standard loss function for regression problems and penalizes larger prediction errors more strongly.

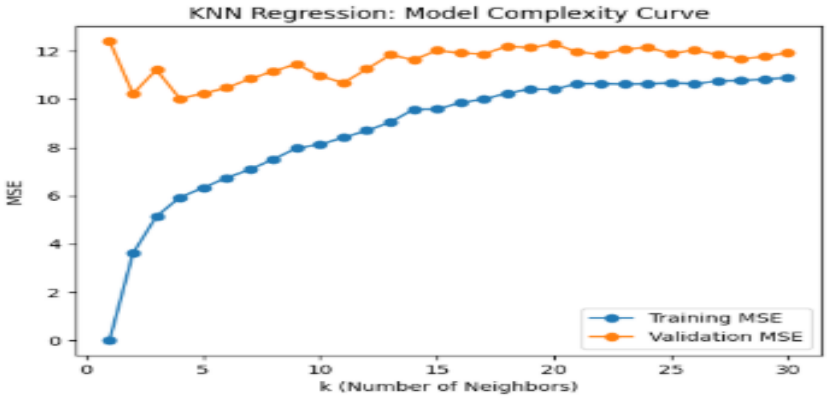

Figure 12: KNN: Train and validation error as a function of k

From the figure, it can be observed that when k is very small, the training error is extremely low while the validation error is relatively higher. This indicates that the model fits the training data very closely and may capture noise in the data, which is associated with high variance and potential overfitting.
As k increases, predictions are averaged over more neighbors, making the model less sensitive to individual training points. Consequently, the training error gradually increases while the validation error becomes more stable.

The value of k also influences the bias and variance of the model.
Small values of k correspond to low bias but high variance.
When k is small, predictions are based on only a few neighboring data points, allowing the model to adapt closely to the training data and capture local patterns, which leads to low bias. However, this also makes the model very sensitive to individual training points and noise in the data, resulting in high variance.
As k increases, predictions are averaged over more neighbors. This reduces the influence of individual data points and therefore decreases the variance. However, averaging over many neighbors makes the model less able to capture local patterns in the data, which increases the bias.

The best value of k was selected based on the lowest validation MSE, since the validation set provides an estimate of the model's performance on unseen data.
Although the figure shows similar validation errors for several values of k, the exact numerical values computed in the code indicate that $k = 4$ achieves the lowest validation MSE. Therefore, $k = 4$ was selected as the final KNN regression model.

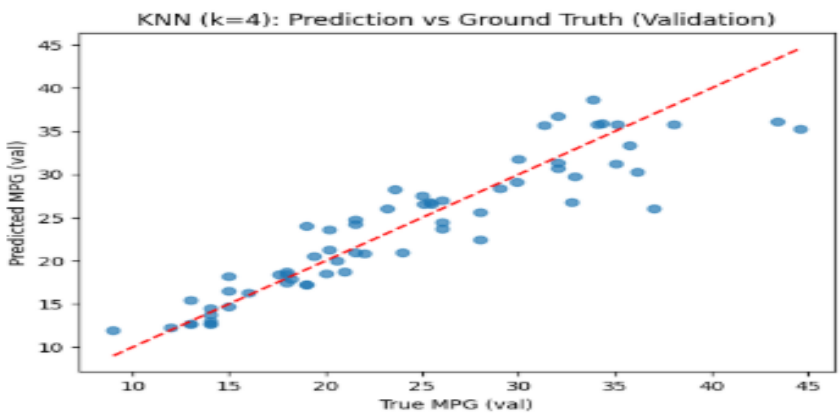

Figure 13: KNN (k=4): Prediction vs Ground Truth

The prediction vs. ground truth plot for $k = 4$ shows that most predictions lie relatively close to the $y = x$ reference line, indicating that the model is able to capture the relationship between the features and the target variable reasonably well.

When KNN Outperforms or Underperforms Parametric Models
In general, KNN can outperform parametric models when the relationship between the explanatory variables and the target variable is complex and cannot be well described by a fixed functional form, such as linear or polynomial models. In such cases, parametric models may be limited because they assume a specific functional structure for the relationship between the variables. In contrast, KNN does not assume such a structure and makes predictions based on nearby data points, allowing it to adapt more flexibly to local patterns in the data.
However, KNN may underperform compared to parametric models when the number of samples is small, when the data is noisy, or when the number of explanatory variables is large. In such situations, predictions based on distances between data points become less reliable.

Optimization Behavior (2.6)

We compare two optimization strategies, Batch Gradient Descent (GD) and Stochastic Gradient Descent (SGD).
In Batch GD, the gradient is computed using the entire training dataset at each iteration, while in SGD the parameters are updated using a single training sample at a time.
During training, the Mean Squared Error (MSE) was recorded for both the training and validation sets across iterations in order to analyze the optimization behavior of the two methods.

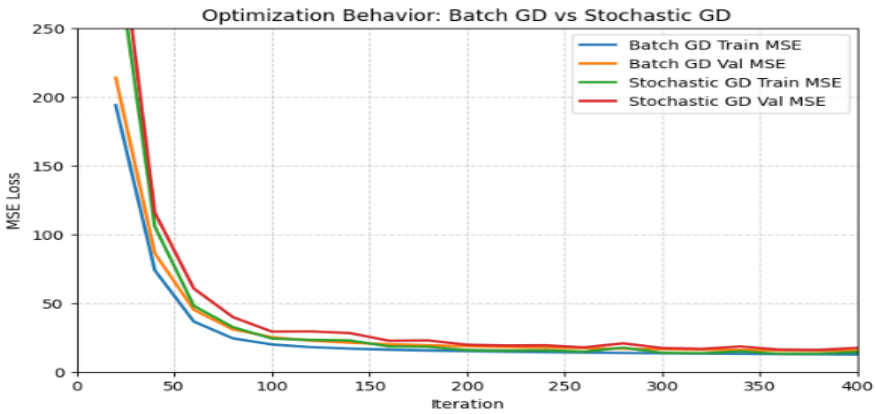

Figure 14: Optimization Behavior: Batch GD vs Stochastic GD

The figure shows the training and validation losses as a function of the number of iterations for two optimization methods: Batch Gradient Descent (Batch GD) and Stochastic Gradient Descent (SGD).
In the first iterations, both the training and validation losses decrease rapidly for both optimization methods. After several iterations, the loss values stabilize and both methods converge to similar values. Batch GD produces smoother and more stable loss curves, while the SGD curves show small fluctuations due to the noise introduced by stochastic updates, since each update relies on a single training sample. In contrast, Batch GD computes the gradient using the entire dataset at each iteration, resulting in smoother updates.

Model Comparison and Discussion (2.7)

To conclude the regression task, we compare the three models studied in this project: Linear Regression, Polynomial Regression, and KNN Regression.

The following table summarizes the validation performance of all models using four standard regression metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2.

Table 2: Comparison of Regression Models on the Validation Set

Model	MSE	RMSE	MAE	R^2
Linear Regression	12.898	3.591	2.796	0.814
Polynomial Regression (deg=2)	11.791	3.434	2.334	0.830
KNN Regression ($k = 4$)	10.033	3.167	2.311	0.856

Predictive performance:
The results in the table show that the KNN regression model with $k = 4$ achieved the best predictive performance on the validation set. It obtained the lowest values for MSE, RMSE, and MAE, and also achieved the highest R^2 score. Lower values of MSE, RMSE, and MAE indicate smaller prediction errors, while a higher R^2 value indicates that the model explains the data better.
Polynomial regression with degree 2 achieved better performance than the linear regression model according to the validation metrics, with lower error values (MSE, RMSE, and MAE) and a higher R^2 score.
This indicates that adding some flexibility to the model improved the predictions. However, its results were still weaker than those of KNN.
Linear regression produced the highest error values, which suggests that it was the least accurate model for this dataset, although it still served as a useful baseline model for comparison, helping evaluate whether more complex models actually improve predictive performance.

Bias - variance characteristics:
The comparison between the models also highlights differences in bias and variance, which are influenced by the complexity of the model.

Linear regression is the simplest model and assumes a purely linear relationship between the features and the target variable. Because of this assumption, the model may not be flexible enough to capture more complex patterns in the data, which leads to higher bias. This is also reflected in the validation results shown in the table, where the linear regression model exhibits higher validation error compared to the other models.

Polynomial regression allows non-linear relationships between the variables, which increases the flexibility of the model. As can be seen in Figure 11, as the polynomial degree increases (i.e., as model complexity increases), the bias of the model typically decreases because the model can fit the training data more closely. However, higher model complexity also increases the variance, making the model more sensitive to the training data.

KNN regression makes predictions based on the values of neighboring data points, where the parameter k determines the number of neighbors used to compute the prediction. As can be seen in Figure 12, the value of k influences both the bias and the variance of the model. Small values of k correspond to low bias but high variance, since predictions are based on only a few neighboring data points. This allows the model to adapt closely to the training data and capture local patterns, but also makes it very sensitive to individual data points and noise. As k increases, predictions are averaged across more neighbors, which reduces the influence of individual samples and therefore decreases the variance. However, averaging over many neighbors makes the model less able to capture local patterns in the data, which increases the bias.

Interpretability:
In terms of interpretability, linear regression is the easiest model to understand and explain. The coefficients of the model directly describe how each feature influences the predicted MPG value. Polynomial regression is somewhat harder to interpret, because the relationship between the variables becomes non-linear and involves additional polynomial terms. KNN regression is generally considered less interpretable than the other models, since it does not produce explicit coefficients or a mathematical formula. Instead, predictions are made based on the similarity between data points in the feature space.

Practical considerations:
From a practical perspective, linear regression is a simple and computationally efficient model, and therefore it is usually fast to train (especially when the number of features is small). It is also relatively easy to implement. Polynomial regression is also relatively easy to implement, but it requires careful selection of the polynomial degree in order to avoid overfitting. KNN regression achieved the best predictive performance in this project, but it also has several disadvantages. The model must store the entire training dataset, and predictions may be slower because distances must be computed between the test sample and many training samples. In addition, KNN is sensitive to feature scaling, which makes proper preprocessing particularly important.

Final model selection:
The final model was selected based on the lowest validation MSE.

MSE was chosen as the main evaluation metric because it is a standard metric for regression problems and penalizes larger prediction errors more strongly. In addition, it was used consistently when comparing the different models and selecting the best hyperparameters (e.g., choosing the value of k in KNN and the polynomial degree in polynomial regression).
According to the validation results, KNN regression with $k = 4$ achieved the lowest validation MSE and was therefore selected as the model for deployment.
After selecting the model, it was retrained on the combined training and validation datasets and evaluated once on the test set. The final test performance was:
MSE = 5.345, RMSE = 2.312, MAE = 1.912, and $R^2 = 0.900$.
These results are consistent with the validation results shown in the comparison table, where KNN regression with $k = 4$ achieved the lowest error values (MSE, RMSE, and MAE) as well as the highest R^2 value among all the models. The test error is even slightly lower than the validation error, which further supports that the model generalizes well to unseen data.
Therefore, although KNN is less interpretable and slightly more computationally demanding than linear regression, it was chosen as the preferred model for deployment in this project because it achieved the best overall predictive performance.

2 Classification Models

In this section, we analyze a supervised multi - class classification problem using the CIFAR-10 dataset. The objective is to classify 32×32 color images into one of ten predefined classes.
This task allows us to evaluate how classical machine learning models perform on high-dimensional image data and to understand their limitations.

The training data was split into two parts: 80% of the data was used as the training set and 20% was used as the validation set. The validation set was used for model comparison and hyperparameter selection.

Before training the models, the features were standardized so that all features are on a similar scale. This is important for models that rely on distances between data points, such as KNN, and also helps models like Logistic Regression and SVM train more effectively.

Baseline Models (3.1)

In this section, we implement and evaluate three classical machine learning models for the CIFAR-10 classification task:

- Multiclass Logistic Regression (Softmax)
- Linear Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

All models were trained using the same train/validation split in order to ensure a fair comparison.
Each model was first trained using default hyperparameters to obtain baseline performance on the validation set (C=1 for Logistic Regression and Linear SVM, and k=5 for KNN).

The goal of this stage is to observe how classical machine learning models perform on high-dimensional image data before performing hyperparameter tuning.

The validation accuracy obtained by the baseline models is reported below:

- Logistic Regression: 0.3732
- Linear SVM: 0.3555
- KNN: 0.3289

Logistic Regression performed best among the baseline models, since it achieved the highest validation accuracy.

Hyperparameter Selection (3.2)

After evaluating the baseline models, we perform hyperparameter tuning for each model in order to improve their performance.
For each model, one main hyperparameter is selected and several possible values are evaluated.
The best value is chosen based on validation accuracy.
The results are presented using plots that show validation accuracy as a function of the hyperparameter.

Logistic Regression

We tuned the regularization strength C.
This parameter controls the strength of regularization: smaller values correspond to stronger regularization, while larger values allow the model to fit the training data more closely.
The following values were evaluated:

$$C \in \{0.01, 0.1, 1, 10\}.$$

The obtained validation accuracies were 0.3984 for $C = 0.01$, 0.3845 for $C = 0.1$, 0.3732 for $C = 1$, and 0.3668 for $C = 10$.
The best performance was achieved at $C = 0.01$, since this value produced the highest validation accuracy.

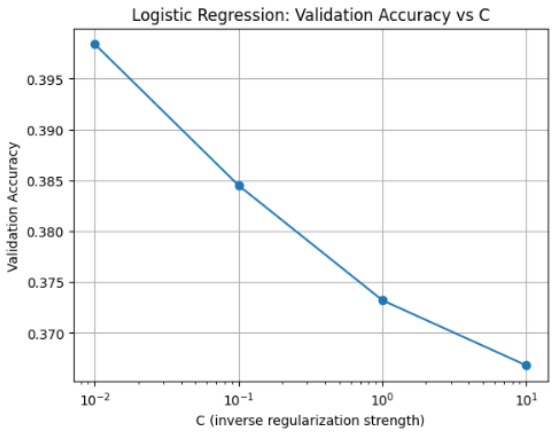

Figure 15: Validation accuracy as a function of C for Logistic Regression

Linear SVM

For the Linear SVM, we also tune the regularization strength C.
This parameter controls the trade-off between maximizing the margin and minimizing classification errors.
The same set of values was evaluated:

$$C \in \{0.01, 0.1, 1, 10\}$$

The obtained validation accuracies were 0.3847 for $C = 0.01$, 0.3694 for $C = 0.1$, 0.3555 for $C = 1$, and 0.3523 for $C = 10$.
The best performance was achieved at $C = 0.01$, since this value produced the highest validation accuracy.

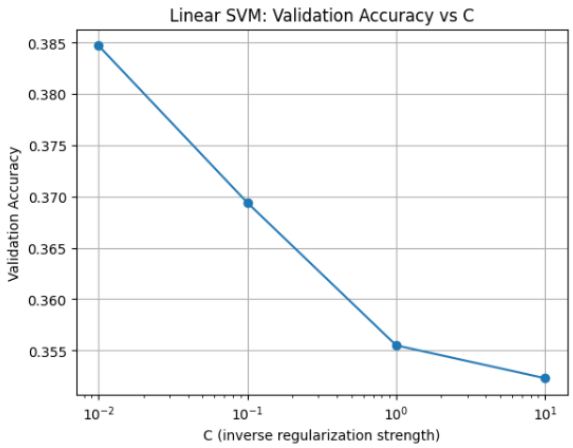

Figure 16: Validation accuracy as a function of C for Linear SVM

KNN

For the K-Nearest Neighbors model we tune the number of neighbors k.
This parameter determines how many neighboring training samples are considered when making a prediction.
The following values were evaluated:

$$k \in \{1, 3, 5, 7, 9, 11, 15\}$$

The obtained validation accuracies were 0.3443 for $k = 1$, 0.3231 for $k = 3$, 0.3289 for $k = 5$, 0.3276 for $k = 7$, 0.3291 for $k = 9$, 0.3259 for $k = 11$, and 0.3259 for $k = 15$.
The best performance was achieved at $k = 1$, since this value produced the highest validation accuracy.

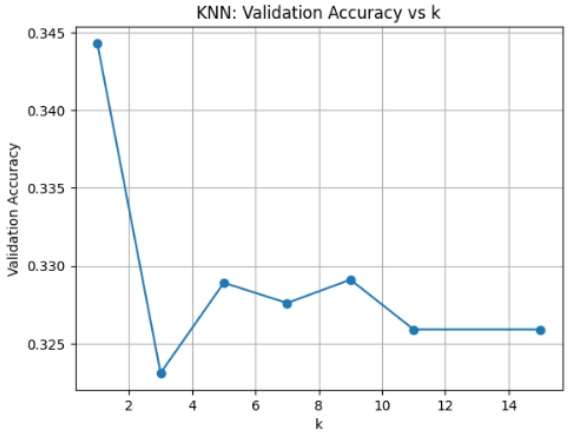

Figure 17: Validation accuracy as a function of k for KNN

Evaluation and Analysis (3.3)

After selecting the best hyperparameter for each model based on validation performance (Section 3.2), we compare the models using their corresponding validation accuracies.

The validation accuracy of each tuned model is reported below:

- Logistic Regression ($C = 0.01$): 0.3984
- Linear SVM ($C = 0.01$): 0.3847
- KNN ($k = 1$): 0.3443

Confusion Matrix Analysis:

Using the models with the selected hyperparameters, we generate the confusion matrices on the validation set to analyze class-specific behavior and identify which classes are more difficult to classify.

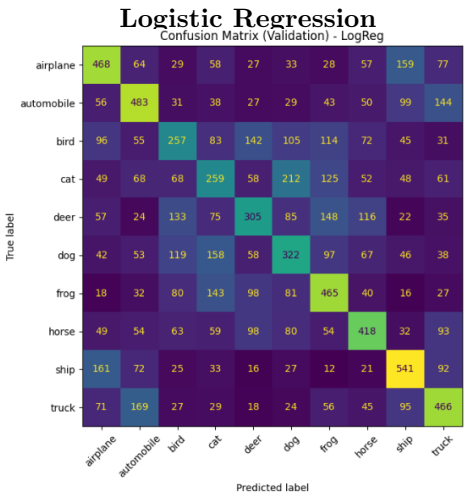

Figure 18: Confusion Matrix for Logistic Regression ($C = 0.01$)

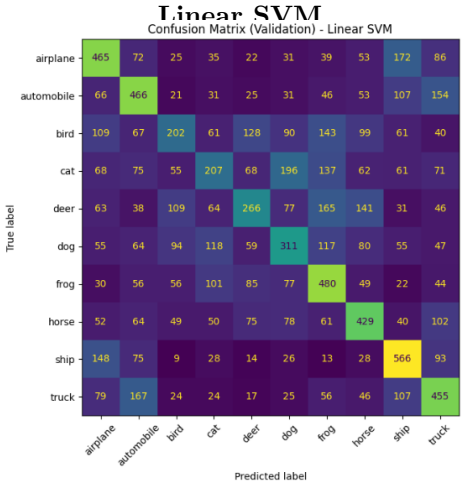

Figure 19: Confusion Matrix for SVM ($C = 0.01$)

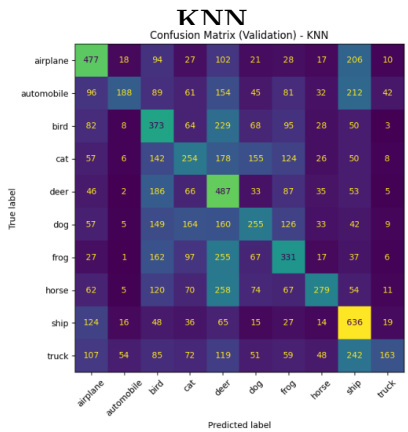

Figure 20: Confusion Matrix for KNN ($k = 1$)

Which models perform better and why:
Logistic Regression achieved the highest validation accuracy, followed by Linear SVM, while KNN obtained the lowest performance.
This trend is also reflected in the confusion matrices.
In the confusion matrices of Logistic Regression, more values appear along the diagonal of the matrix, meaning that in more cases the predicted class matches the true class. Since the diagonal entries represent correct classifications, a larger number of values along the diagonal indicates that the model makes more correct predictions and therefore achieves a higher overall accuracy.
In the confusion matrix of Linear SVM, diagonal values are also visible, but compared to Logistic Regression there are slightly more values outside the diagonal, indicating a somewhat higher level of confusion between similar classes. In contrast, the confusion matrix of KNN contains even more values outside the diagonal, meaning that the model produces more incorrect predictions, which is consistent with its lower validation accuracy.

The effect of high input dimensionality:
In the CIFAR-10 dataset, each image is represented by 3072 numerical features (32×32 pixels with three color channels), which creates a high dimensional input representation.
High dimensional data can make it difficult for classical machine learning models to identify meaningful patterns in the data. In particular, distance based methods such as KNN are especially sensitive to high dimensional feature spaces, since distance measures become less effective at distinguishing between different samples when the number of features is large.
This may explain why KNN achieved the lowest validation accuracy among the tested models and why more misclassifications can be observed in its confusion matrix.

In contrast, the models Logistic Regression and Linear SVM can handle high dimensional data more efficiently, which may explain why they achieved higher validation accuracy. However, these models still have limitations, since they treat the image as a long vector of numerical values rather than learning the internal structure of the image.

Limitations of classical models on image data:
In general, classical machine learning models treat an image as a long vector of numerical values and do not take into account the relationships between neighboring pixels in the image. As a result, they have difficulty identifying complex visual patterns, such as edges, shapes, or parts of objects that appear in the image.
This limitation is also reflected in the confusion matrices. For example, it can be observed in all confusion matrices of the classical models (Logistic Regression, Linear SVM, and KNN) that there is significant confusion between the classes cat and dog, since cats and dogs share similar visual characteristics, such as a similar body structure, four legs, and fur, and therefore the models have difficulty distinguishing between them.

Final Model and Test Evaluation:
The model with the highest validation accuracy was Logistic Regression. Therefore, Logistic Regression was selected as the final model.
The selected model was retrained using the combined training and validation data.
The model was then evaluated once on the test set to report the final classification accuracy.
The final test accuracy achieved by Logistic Regression is 0.397.

3 Neural Network Classification (PyTorch)

In this part of the project, we implemented a fully connected neural network in PyTorch for the CIFAR-10 classification task.
CIFAR-10 images are RGB images of size 32×32. Therefore, each image was first flattened into a vector of size 3072 ($3 \times 32 \times 32$), which was used as the input to the neural network.

Architecture Design and Hyperparameter Selection:
We built the model gradually and examined several different options during the experiments. Instead of selecting the hyperparameters randomly, we compared several alternatives in different stages, where in each stage the option that achieved the highest Validation Accuracy was selected.

In the first stage we tested different network structures. **After that** we compared different activation functions. **Next**, we evaluated different optimizers, **then** we compared different learning rates, **and finally** we tested different dropout values.

During the hyperparameter selection process, all models that were tested were trained for the same number of epochs (10) in order to allow a fair comparison between the different options.

Training was performed using mini-batches with a batch size of 64, as defined earlier in the code in the notebook. During training, the model was trained on the training set.

Model selection and hyperparameter tuning were performed using the validation set.
The test set was not used during the tuning process and was evaluated only once at the end of the process in order to report the final performance of the model.

We started by comparing several different network structures. Specifically, we tested the following hidden layer configurations:

- $(512,)$
- $(1024, 512)$
- $(1024, 512, 256)$

The reason we started by comparing different network structures is that the number of layers and their sizes influence the model's ability to learn patterns in the data. A model with too few parameters may be too simple and fail to learn the patterns properly, while a deeper model can learn more complex representations but may also lead to overfitting.
The results obtained were:

- (512) - Validation accuracy: 49.6
- $(1024, 512)$ - Validation accuracy: 55.2
- $(1024, 512, 256)$ - Validation accuracy: 54.4

According to these results, the selected architecture is: (1024, 512).

The next step we examined was the comparison of different activation functions.
This stage was performed after selecting the network architecture, since once the basic architecture of the model is fixed, we can examine how the type of activation function affects the model's performance for the same architecture.
In this stage we compared the following activation functions:

- ReLU
- Tanh
- Sigmoid

The results obtained were:

- ReLU - Validation accuracy: 50.4

- Tanh - Validation accuracy: 40.7
- Sigmoid - Validation accuracy: 49.7

According to these results, the selected activation function is: ReLU.

In the next stage we compared different optimizers.
This stage was performed after selecting the activation function because the optimizer affects the training process itself, and therefore it is important to evaluate it when the model architecture and activation function are already fixed.
The optimizer determines how the model parameters are updated during training, and therefore choosing an appropriate optimizer can significantly influence the convergence speed and the final performance of the model.
In this stage we compared the following optimizers:

- SGD
- SGD with Momentum
- Adam

The results obtained were:

- SGD - Validation accuracy: 44.3
- SGD with Momentum - Validation accuracy: 55.4
- Adam - Validation accuracy: 54.5

According to these results, the selected optimizer is: SGD with Momentum.

In the next stage we compared different learning rates.
The learning rate determines the step size used to update the model parameters during training, and therefore it has a significant influence on the stability of the training process and the speed of convergence.
In this stage we tested the following values:

- 0.01
- 0.001
- 0.0005

The results obtained were:

- 0.01 - Validation accuracy: 52.3
- 0.001 - Validation accuracy: 55.2
- 0.0005 - Validation accuracy: 52.5

According to these results, the selected learning rate is: 0.001.

Finally, we tested different dropout values.
This stage was performed at the end of the selection process because dropout is a regularization mechanism intended to reduce overfitting after the other components of the model have already been determined.
Dropout works by randomly disabling a subset of neurons during training, which helps the model learn more robust representations and improves its ability to generalize.
In this stage we compared the following dropout values:

- 0.0
- 0.3
- 0.5

The results obtained were:

- 0.0 - Validation accuracy: 54.6
- 0.3 - Validation accuracy: 54.4
- 0.5 - Validation accuracy: 54.3

According to these results, the selected dropout value is: 0.0.

Final Model Configuration and Test Performance:
Based on the validation results obtained in the previous stages, the final configuration of the model was:

- Architecture: (1024, 512)
- Activation function: ReLU
- Optimizer: SGD with Momentum
- Learning rate: 0.001
- Dropout: 0.0

The final validation accuracy achieved by the selected configuration was:
Final validation accuracy: 54.4
After the hyperparameter tuning process was completed, the final model was evaluated on the test set. The test set was used only once at the end of the process after all hyperparameter tuning was completed, in order to measure the model's performance on previously unseen data.
The final test accuracy achieved by the model was:
Final test accuracy: 53.85

What we learned from the comparisons:
From these experiments we learned several important things about neural network design:

First, increasing the depth of the network improved performance when moving from one hidden layer to two hidden layers. However, adding a third hidden layer did not lead to further improvement.
This finding suggests that increasing the depth of the network can improve performance up to a certain point. We believe that this is because as the network becomes deeper, the training process may become more complex, and therefore additional depth does not necessarily lead to further improvement in performance.

Second, the activation function also affected the model's performance.
In our experiments, ReLU achieved the highest validation accuracy among the activation functions that were examined. We believe that this function was selected because it behaved more stably during training and allowed the model to learn more efficiently, which may explain why it produced the best results in our experiments.

Third, the choice of optimizer influenced both the learning speed and the overall effectiveness of the training process.
From the results we observed that SGD with momentum achieved the highest validation accuracy. Adam also performed well but achieved slightly lower performance, while SGD without momentum obtained the lowest validation accuracy among the three options.
A possible explanation is that SGD with momentum also uses information from previous parameter updates, which allows the optimizer to progress in a more stable and consistent way during training. In contrast, standard SGD relies only on the gradient of the current step, and therefore the learning process may be less stable and sometimes progress more slowly. Adam also performed well because it adapts the learning rate of the parameters during training, which often helps the model converge efficiently. However, in our experiments it appears that the combination of more stable updates together with the selected learning rate allowed SGD with momentum to achieve the best results.

Fourth, the learning rate also had a significant effect on the model's performance.
From the comparison between different learning rate values, we observed that an appropriate learning rate allowed the model to learn more stably and reach higher validation accuracy, while values that were too large or too small negatively affected performance.

Finally, the dropout experiments showed that adding dropout did not improve the model's performance in our experiments.
The best validation accuracy was achieved when no dropout was used (dropout = 0.0), while higher dropout values slightly reduced the validation accuracy. This suggests that additional regularization was not necessary for this model and training setup.

Overall, the experiments we conducted show that making careful and well-supported choices of hyperparameters can have a significant impact on the performance of neural network models.

4 Neural Network Regression (PyTorch)

In this bonus task, we adapted the PyTorch notebook from Part III, which was used for CIFAR-10 classification, in order to implement a neural network to solve the regression problem from Part I using the Auto MPG dataset.
The goal of the model is to predict a car's fuel efficiency (MPG) based on vehicle characteristics.

As in Part I, the feature car_name was excluded, since it is a textual identifier and is not useful for prediction.
The data was first split into training, validation, and test sets.
Preprocessing was fitted on the training set and then applied to the validation and test sets.
The preprocessing pipeline included standardization of numerical features, mean imputation for missing values in horsepower, and one-hot encoding for the categorical feature origin.
After preprocessing, the input dimension of the neural network was 9 features.

Neural Network Architecture and Hyperparameter Selection

Unlike the classification model from Part III, which produced scores for the 10 CIFAR-10 classes, the regression model used here predicts a single continuous value (MPG). Therefore, the output layer of the network contains a single neuron instead of a multi-class output.

Model selection and hyperparameter tuning were examined using the validation set. In this process, several different configurations were tested, including different network architectures, activation functions, optimizers, learning rates, and dropout values.
At each stage, the configuration that achieved the lowest validation loss was selected.
The final configuration chosen based on validation performance was:

- Architecture: (128, 64, 32)
- Activation function: ReLU
- Optimizer: SGD with Momentum
- Learning rate: 0.001
- Dropout: 0.0

After selecting the final model using the validation set, the model was retrained on the combined training and validation data and then evaluated on the test set.
The test set was used only once at the end of the process for the final evaluation of the model's performance.
The final test performance of the neural network regression model was:

- $MSE = 5.6847$
- $RMSE = 2.3843$
- $R^2 = 0.8943$

Loss Function and Evaluation

For this regression task, we used Mean Squared Error (MSE) as the loss function.
MSE is a standard loss function for regression problems because it measures the average squared difference between the predicted values and the true target values, and it penalizes larger prediction errors more strongly.

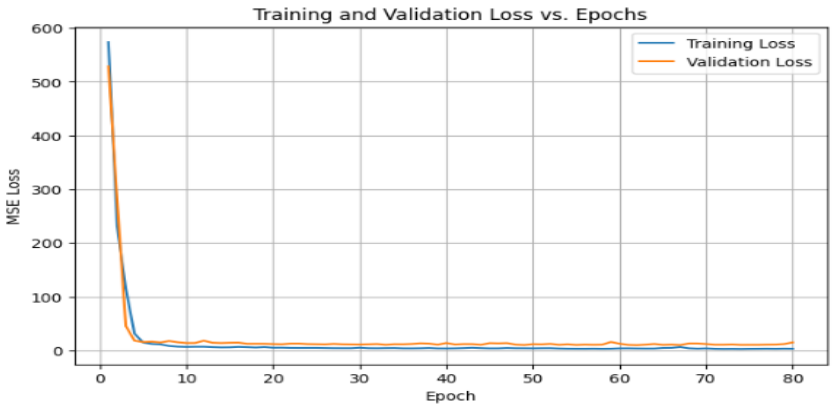

Figure 21: Training and validation MSE as a function of epochs

From the figure, it can be seen that both the training loss and the validation loss decrease sharply during the first epochs, indicating that the model quickly learns meaningful patterns from the data.
After the initial epochs, both curves stabilize and fluctuate slightly, suggesting that the model has converged.
The validation loss remains close to the training loss, indicating that the model generalizes reasonably well without severe overfitting.

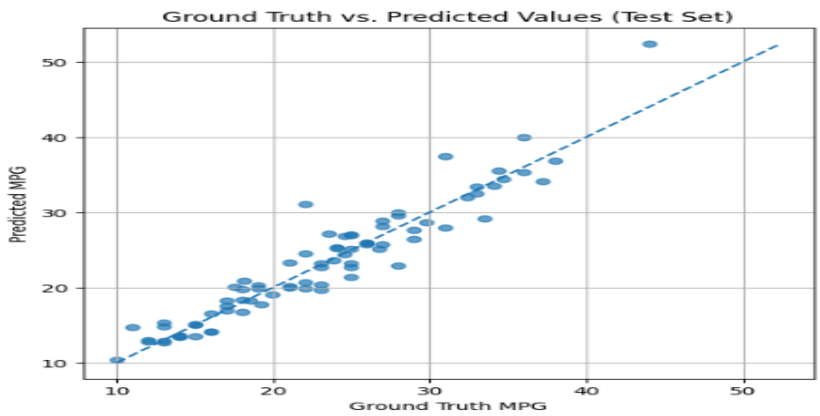

Figure 22: Ground truth MPG vs. predicted MPG

It can be seen that most of the points lie close to the diagonal line, indicating that the model is able to capture the general relationship between the input features and the target variable.
However, some points are located at a certain distance from the line, which suggests prediction errors for certain samples.

Comparison with the Best Regression Model from Part I

In Part I, the best regression model was KNN regression with $k = 4$.
Its final test performance was:

- MSE = 5.345
- RMSE = 2.312
- $R^2 = 0.900$

Compared with this result, the neural network regression model achieved:

- MSE = 5.6847
- RMSE = 2.3843
- $R^2 = 0.8943$

The neural network achieved performance that is very close to that of the KNN model, but KNN obtained slightly better results. This is because it achieved lower error values as well as a slightly higher R^2 score on the test set.

One possible explanation is that the Auto MPG dataset is relatively small. In such situations, models like KNN can work well because they are based on computing distances between similar samples in the data.
Neural networks, on the other hand, usually perform better when larger datasets are available, which allow them to learn more complex patterns more effectively.

5 Code and Implementation

The full implementation of this project, including all notebooks and scripts used to reproduce the reported results, is available in the following GitHub repository:

`https://github.com/noakohn/supervised-learning-final-project`